

Proyecto 3: Juego competitivo de Othello utilizando Alpha-Beta Pruning, Monte Carlo Tree Search y Heurísticas Adaptativas

MACHIC D.¹, (mac22118@uvg.edu.gt), RAMÍREZ M.⁴, (ram221051@uvg.edu.gt) DIAZ M.⁵, (dia221102@uvg.edu.gt)

¹Universidad del Valle de Guatemala

Inteligencia Artificial

Instructor: Cat. Alan Reyes Figueroa

ABSTRACT

Este proyecto desarrolla un sistema inteligente para el juego Othello (Reversi) mediante la implementación de agentes basados en Minimax, Alpha-Beta Pruning y Monte Carlo Tree Search (MCTS), apoyados por una función heurística adaptativa que considera movilidad, control de esquinas, bordes y ventaja material. Se construyó además un motor completo del juego y una interfaz gráfica en Pygame para la interacción y visualización de partidas. La evaluación experimental, realizada mediante torneos automáticos, mostró que Alpha-Beta obtuvo el mejor desempeño con 17 victorias en 20 partidas frente a MCTS, además de requerir menor costo computacional. Asimismo, se comprobó que la poda Alpha-Beta reduce significativamente la cantidad de nodos explorados respecto a Minimax sin afectar la calidad de las decisiones, demostrando la efectividad de las técnicas de búsqueda adversarial y evaluación heurística en juegos de estrategia de información perfecta.

INDEX TERMS

Artificial Intelligence, Othello, Reversi, Minimax, Alpha-Beta Pruning, Monte Carlo Tree Search, Heuristic Evaluation, Adversarial Search, Game Playing.

I. ANTECEDENTES

El juego Othello (Reversi) es un problema clásico de Inteligencia Artificial debido a la gran cantidad de estados que pueden generarse durante una partida. A pesar de sus reglas simples, su complejidad estratégica requiere algoritmos capaces de anticipar movimientos futuros y seleccionar acciones óptimas dentro de un amplio espacio de búsqueda. En este contexto, algoritmos como Minimax permiten determinar la mejor jugada asumiendo un comportamiento racional de ambos jugadores.

No obstante, el crecimiento exponencial del árbol de búsqueda limita la profundidad de exploración, por lo que se emplean técnicas como Alpha-Beta Pruning para reducir la cantidad de estados evaluados sin afectar la calidad de las decisiones. De forma complementaria, Monte Carlo Tree Search (MCTS) utiliza simulaciones para estimar la calidad de los movimientos, ofreciendo una alternativa eficiente para enfrentar espacios de búsqueda complejos.

La efectividad de estos algoritmos depende en gran medida de la evaluación de estados no terminales mediante funciones heurísticas que consideran factores estratégicos como movilidad, control de esquinas, control de bordes y ventaja material. El análisis comparativo de estas técnicas permite estudiar su desempeño en términos de calidad de decisión, tiempo de ejecución y complejidad de exploración, contribuyendo al desarrollo de agentes inteligentes para juegos de estrategia de información perfecta.

II. OBJETIVOS

II-A. OBJETIVO GENERAL

Implementar y comparar agentes inteligentes para el juego Othello utilizando algoritmos de búsqueda adversarial y búsqueda basada en simulaciones, analizando su desempeño mediante métricas de exploración, tiempo de decisión y resultados de juego.

II-B. OBJETIVOS ESPECÍFICOS

- Implementar la lógica completa del juego Othello y desarrollar una interfaz gráfica para visualizar el tablero e interactuar con jugadores humanos y agentes inteligentes.
- Desarrollar agentes basados en Minimax, Alpha-Beta Pruning y Monte Carlo Tree Search (MCTS), incorporando una función heurística que considere movilidad, control de esquinas, control de bordes y fase de la partida.
- Evaluar y comparar el desempeño de los algoritmos mediante métricas como tiempo de búsqueda, cantidad de nodos explorados, número de simulaciones y resultados obtenidos en partidas automáticas.

III. HERRAMIENTAS Y METODOLOGÍA

III-A. HERRAMIENTAS UTILIZADAS

El proyecto fue desarrollado en Python utilizando diferentes librerías orientadas a la implementación de algoritmos de búsqueda adversarial, simulación de partidas y visualización gráfica del juego.

- **NumPy**: utilizado para la representación y manipulación eficiente del tablero de Othello mediante arreglos bidimensionales.
- **Pygame**: empleado para desarrollar la interfaz gráfica del juego, permitiendo la visualización del tablero, las fichas y la interacción con los agentes inteligentes.
- **Pandas**: utilizado para almacenar y presentar los resultados obtenidos durante los torneos automáticos entre agentes.
- **Matplotlib**: empleado para generar gráficas comparativas relacionadas con victorias, tiempos de búsqueda y complejidad de exploración.
- **Math**: utilizado para operaciones matemáticas requeridas por los algoritmos de búsqueda y la fórmula UCT implementada en MCTS.
- **Random**: empleado para seleccionar movimientos aleatorios durante las simulaciones realizadas por Monte Carlo Tree Search. ““

III-B. REPRESENTACIÓN DEL JUEGO

El juego Othello fue modelado mediante una matriz de tamaño 8×8 , donde cada celda representa una posición del tablero. Se utilizaron valores enteros para identificar espacios vacíos y fichas pertenecientes a cada jugador.

La lógica principal del juego fue implementada en una clase encargada de validar movimientos, determinar fichas capturadas, aplicar reglas oficiales de volteo y detectar estados terminales. Asimismo, se incorporaron métodos auxiliares para generar movimientos legales, contabilizar fichas y determinar el ganador de la partida.

III-C. FUNCIÓN HEURÍSTICA

Debido a la imposibilidad práctica de explorar completamente el árbol de juego, se implementó una función heurística para evaluar estados no terminales durante la búsqueda.

La evaluación considera diferentes características estratégicas del tablero:

- **Movilidad**: diferencia entre la cantidad de movimientos legales disponibles para cada jugador.
- **Control de esquinas**: evaluación de las cuatro esquinas del tablero, consideradas posiciones de alto valor estratégico.
- **Control de bordes**: cantidad de fichas ubicadas en los bordes del tablero.
- **Posiciones X y C**: casillas cercanas a las esquinas cuyo control puede resultar favorable o desfavorable dependiendo del contexto de juego.
- **Diferencia de fichas**: ventaja material entre ambos jugadores.

Tabla 1: Pesos utilizados en la función heurística

Componente	Apertura	Medio Juego	Final
Movilidad	5	4	2
Control de esquinas	30	40	40
Control de bordes	3	5	3
Posiciones X	10	8	4
Posiciones C	8	6	2
Diferencia de fichas	1	2	10

Tabla 2: Fases de la heurística

Fase	Criterio Principal
Apertura (< 20)	Movilidad
Medio Juego ($20 - 49$)	Esquinas y bordes
Final (≥ 50)	Diferencia de fichas

Además, los pesos asignados a cada componente fueron ajustados dinámicamente según la fase de la partida (apertura, medio juego y final determinadas por la cantidad de movimientos que lleven los algoritmos), permitiendo aproximar estrategias utilizadas en jugadores competitivos.

III-D. ALGORITMOS DE BÚSQUEDA

Para la toma de decisiones se implementaron diferentes algoritmos de búsqueda utilizados comúnmente en juegos adversariales.

- **Minimax**: explora el árbol de juego asumiendo que ambos jugadores actúan racionalmente y buscan maximizar su utilidad.
- **Alpha-Beta**: extensión de Minimax que incorpora poda de ramas irrelevantes, reduciendo significativamente la cantidad de estados evaluados.
- **Monte Carlo Tree Search (MCTS)**: algoritmo basado en simulaciones que estima la calidad de los movi-

mientos mediante exploración estadística del espacio de búsqueda.

En el caso de MCTS, se implementó la estrategia UCT (*Upper Confidence Bound applied to Trees*) para equilibrar exploración y explotación durante la construcción del árbol de búsqueda.

III-E. INTERFAZ GRÁFICA

Se desarrolló una interfaz gráfica utilizando Pygame con el propósito de visualizar el estado del tablero y facilitar la interacción entre jugadores humanos y agentes inteligentes.

La interfaz permite observar las fichas en tiempo real, los movimientos válidos disponibles, el puntaje actual de cada jugador y métricas relacionadas con el proceso de búsqueda, incluyendo tiempo de decisión y cantidad de nodos explorados.

Asimismo, se implementaron diferentes modos de juego que permiten enfrentar jugadores humanos contra agentes Alpha-Beta o MCTS, así como ejecutar partidas completamente automáticas entre agentes inteligentes.

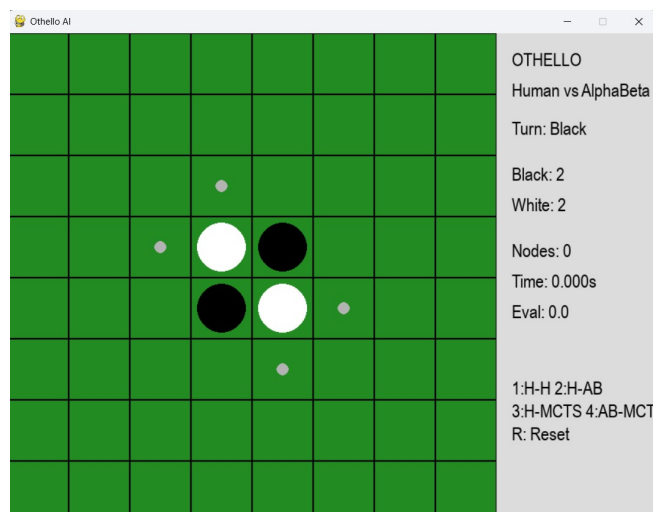


Figura 1: Interfaz gráfica del juego.

III-F. METODOLOGÍA EXPERIMENTAL

Para evaluar el desempeño de los algoritmos implementados, se realizaron partidas automáticas entre los agentes Alpha-Beta y Monte Carlo Tree Search bajo las mismas condiciones iniciales.

Durante cada partida se registraron métricas relacionadas con el resultado final, tiempo promedio de búsqueda y complejidad de exploración. Adicionalmente, se analizaron las diferencias entre Minimax y Alpha-Beta mediante la cantidad de nodos explorados a distintas profundidades de búsqueda.

Los resultados obtenidos fueron organizados en tablas y visualizados mediante gráficas comparativas, permitiendo

analizar la efectividad y eficiencia de cada enfoque para la toma de decisiones en Othello.

IV. RESULTADOS

IV-A. TORNEO ALPHA-BETA VS MCTS

Con el objetivo de evaluar el desempeño de los agentes implementados, se ejecutó un torneo automático de 20 partidas entre Alpha-Beta y Monte Carlo Tree Search (MCTS). Durante cada encuentro ambos algoritmos jugaron bajo las mismas condiciones iniciales, registrando el resultado final y diferentes métricas relacionadas con el proceso de búsqueda.

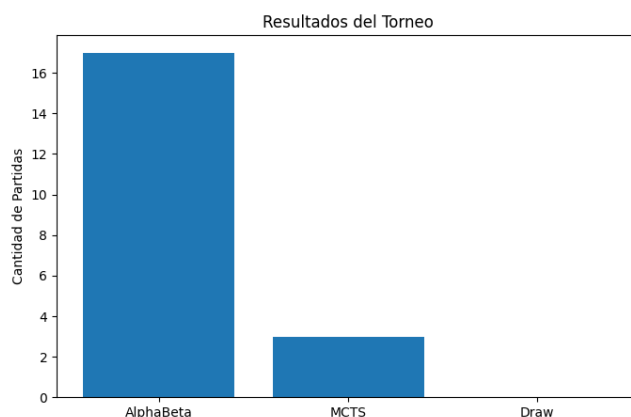


Figura 2: Distribución de victorias obtenidas por Alpha-Beta y MCTS durante el torneo.

La **Figura 2** muestra la cantidad de victorias obtenidas por cada agente durante las 20 partidas realizadas. Se observa una clara superioridad del algoritmo Alpha-Beta frente a MCTS bajo la configuración utilizada.

Tabla 3: Resultados generales del torneo

Resultado	Cantidad
Victorias Alpha-Beta	17
Victorias MCTS	3
Empates	0

La **Tabla 1** evidencia que Alpha-Beta obtuvo el 85 % de las victorias, mientras que MCTS únicamente consiguió el 15 % de los triunfos. No se registraron partidas empatadas durante el experimento.

Los resultados sugieren que la combinación de búsqueda sistemática mediante poda Alpha-Beta y la función heurística diseñada para Othello proporciona decisiones más consistentes que las obtenidas mediante simulaciones aleatorias de Monte Carlo bajo el presupuesto temporal establecido.

IV-B. TIEMPO PROMEDIO DE BÚSQUEDA

Además de analizar los resultados de las partidas, se evaluó el tiempo promedio requerido por cada algoritmo para seleccionar movimientos durante el torneo.

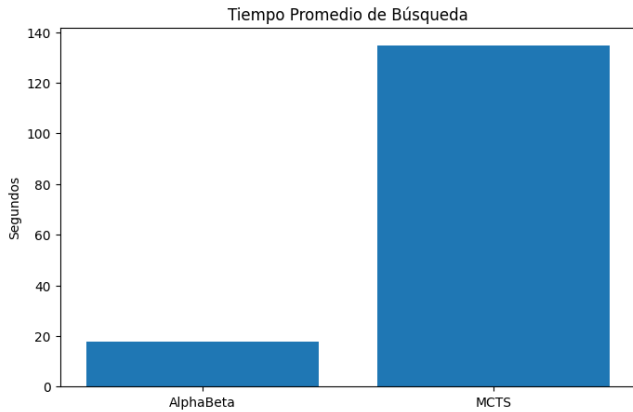


Figura 3: Comparación del tiempo promedio de búsqueda entre Alpha-Beta y MCTS.

La **Figura 3** presenta el tiempo promedio invertido por cada agente durante la toma de decisiones.

Tabla 4: Tiempo promedio de búsqueda

Algoritmo	Tiempo Promedio (s)
Alpha-Beta	17.87
MCTS	134.87

La **Tabla 2** muestra que MCTS requirió significativamente más tiempo de cómputo que Alpha-Beta. Esto se debe a que el algoritmo realiza una gran cantidad de simulaciones para estimar la calidad de cada movimiento, mientras que Alpha-Beta aprovecha la función heurística y la poda para reducir considerablemente el espacio de búsqueda.

IV-C. PROFUNDIDAD DE BÚSQUEDA Y NODOS EXPLORADOS

Para analizar la complejidad computacional del algoritmo Alpha-Beta, se realizaron pruebas variando la profundidad máxima de exploración y registrando la cantidad de nodos evaluados durante la búsqueda.

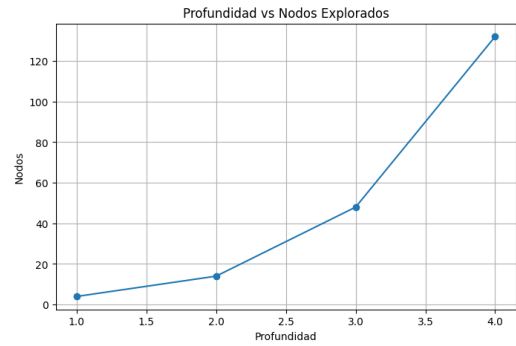


Figura 4: Relación entre profundidad de búsqueda y cantidad de nodos explorados.

La **Figura 4** muestra cómo la cantidad de nodos explorados aumenta conforme se incrementa la profundidad de búsqueda. Este comportamiento es consistente con la naturaleza exponencial de los árboles de decisión utilizados en juegos adversariales.

A pesar de este crecimiento, la poda Alpha-Beta permite reducir significativamente el número de estados evaluados respecto a una búsqueda Minimax tradicional, manteniendo la misma calidad de decisión.

IV-D. FACTOR DE RAMIFICACIÓN EFECTIVO

Con base en la cantidad de nodos explorados a distintas profundidades, se estimó el factor de ramificación efectivo del árbol de búsqueda.

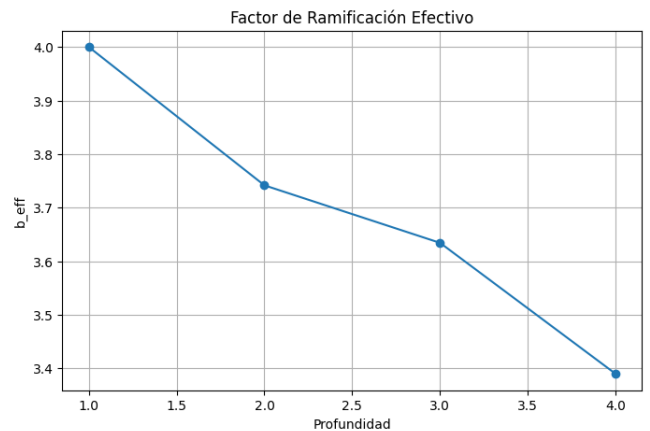


Figura 5: Factor de ramificación efectivo obtenido para diferentes profundidades.

La **Figura 5** presenta la evolución del factor de ramificación efectivo conforme aumenta la profundidad de búsqueda. Esta métrica permite estimar el número promedio de decisiones disponibles en cada nivel del árbol y proporciona una aproximación a la complejidad real del problema.

Los resultados obtenidos muestran que la poda reduce el crecimiento efectivo del árbol, permitiendo explorar mayores

profundidades sin un incremento proporcional en el costo computacional.

IV-E. COMPARACIÓN ENTRE MINIMAX Y ALPHA-BETA

Finalmente, se comparó directamente el desempeño de Minimax y Alpha-Beta utilizando diferentes profundidades de búsqueda.

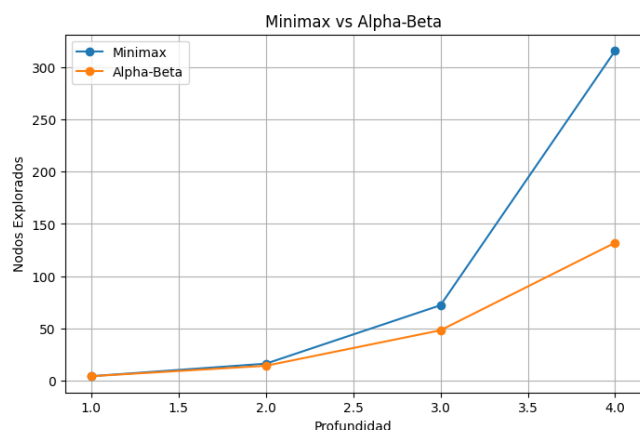


Figura 6: Comparación de nodos explorados entre Minimax y Alpha-Beta.

La **Figura 6** muestra la cantidad de nodos explorados por ambos algoritmos a medida que aumenta la profundidad de búsqueda.

Puede observarse que Minimax presenta un crecimiento considerablemente mayor debido a que explora exhaustivamente todas las ramas posibles del árbol. Por el contrario, Alpha-Beta elimina ramas que no pueden afectar la decisión final, reduciendo significativamente la cantidad de estados evaluados.

Esta diferencia se vuelve más evidente conforme aumenta la profundidad, demostrando que la poda Alpha-Beta constituye una optimización fundamental para aplicar búsqueda adversarial en juegos con espacios de estados complejos como Othello.

V. DISCUSIÓN DE RESULTADOS

Los resultados obtenidos durante el desarrollo del proyecto permitieron evaluar el comportamiento de diferentes técnicas de búsqueda aplicadas al juego Othello. A partir de las partidas automáticas realizadas entre los agentes Alpha-Beta y Monte Carlo Tree Search (MCTS), fue posible analizar tanto la calidad de las decisiones tomadas como el costo computacional asociado a cada enfoque.

El torneo automático mostró una ventaja significativa para el agente Alpha-Beta, que obtuvo 17 victorias de un total de 20 partidas. Este comportamiento puede atribuirse principalmente a la función heurística implementada, la cual incorpora información estratégica relevante del juego, como movilidad, control de esquinas, control de bordes y ventaja material. Gracias a esta evaluación, el algoritmo fue capaz de identificar

posiciones favorables incluso sin explorar completamente el árbol de juego.

Por otra parte, aunque MCTS logró obtener algunas victorias, su desempeño general fue inferior al observado en Alpha-Beta. Una posible explicación es que el algoritmo depende de simulaciones aleatorias para estimar la calidad de los movimientos. Si bien esta estrategia resulta efectiva en problemas donde no existe una heurística confiable, en este caso compitió contra un agente que dispone de conocimiento estratégico explícito sobre el dominio del juego. Como consecuencia, las simulaciones realizadas por MCTS no siempre produjeron estimaciones tan precisas como las evaluaciones heurísticas utilizadas por Alpha-Beta.

Los resultados relacionados con el tiempo de búsqueda también evidenciaron diferencias importantes entre ambos enfoques. MCTS requirió una mayor cantidad de tiempo computacional debido a la ejecución continua de simulaciones durante cada turno. En contraste, Alpha-Beta aprovechó la poda de ramas irrelevantes para reducir significativamente el número de estados evaluados. Esto permitió obtener decisiones competitivas con un costo computacional considerablemente menor.

El análisis de profundidad de búsqueda mostró que la cantidad de nodos explorados aumenta rápidamente conforme se incrementa el horizonte de exploración. Este comportamiento es esperado en juegos de información perfecta como Othello, donde cada posición puede generar múltiples movimientos legales. Sin embargo, los resultados demuestran que la poda Alpha-Beta logra controlar parcialmente este crecimiento, permitiendo explorar niveles más profundos sin que el costo computacional se vuelva prohibitivo.

La comparación directa entre Minimax y Alpha-Beta confirmó experimentalmente uno de los principales fundamentos teóricos de la búsqueda adversarial: ambas técnicas pueden alcanzar la misma decisión óptima, pero Alpha-Beta requiere evaluar una cantidad significativamente menor de estados. Esta diferencia se vuelve cada vez más relevante conforme aumenta la profundidad de búsqueda, lo que explica por qué la poda Alpha-Beta es ampliamente utilizada en sistemas de juego competitivos.

Asimismo, el análisis del factor de ramificación efectivo permitió observar cómo la complejidad del árbol de búsqueda evoluciona a diferentes profundidades. Los resultados sugieren que la reducción de ramas mediante poda contribuye a disminuir el crecimiento efectivo del espacio de estados, favoreciendo una utilización más eficiente de los recursos computacionales disponibles.

Es importante señalar que los resultados dependen de los parámetros seleccionados para cada algoritmo. En particular, el rendimiento de MCTS podría mejorar aumentando el tiempo disponible para las simulaciones o incorporando estrategias de simulación más sofisticadas. De manera similar, Alpha-Beta podría beneficiarse de funciones heurísticas aún

más elaboradas o mecanismos de ordenamiento de movimientos que incrementen la efectividad de la poda.

Se concluye que Alpha-Beta fue el algoritmo más efectivo para las condiciones evaluadas en este proyecto. La combinación de búsqueda adversarial, poda eficiente y evaluación heurística produjo agentes capaces de tomar decisiones consistentes y competitivas dentro de un tiempo razonable. Al mismo tiempo, los resultados muestran que Monte Carlo Tree Search continúa siendo una alternativa válida para problemas donde el conocimiento heurístico es limitado o difícil de modelar explícitamente.

VI. CONCLUSIONES

- Se implementó un sistema completo para el juego Othello, integrando la lógica del juego, agentes inteligentes, funciones heurísticas y una interfaz gráfica para la visualización e interacción de partidas.
- El algoritmo Alpha-Beta obtuvo el mejor desempeño experimental, logrando 17 victorias de 20 partidas frente a Monte Carlo Tree Search (MCTS), lo que demuestra la efectividad de la búsqueda adversarial guiada por heurísticas.
- La función heurística basada en movilidad, control de esquinas, control de bordes y fase de la partida contribuyó significativamente a mejorar la calidad de las decisiones tomadas por el agente Alpha-Beta.
- Los resultados evidencian que la poda Alpha-Beta reduce considerablemente la cantidad de nodos explorados respecto a Minimax, permitiendo búsquedas más profundas y eficientes sin comprometer la calidad de las decisiones.

VII. RECOMENDACIONES

- Incrementar la profundidad de búsqueda e incorporar técnicas de ordenamiento de movimientos para mejorar la eficiencia y el desempeño del algoritmo Alpha-Beta.
- Desarrollar versiones avanzadas de Monte Carlo Tree Search (MCTS) utilizando simulaciones guiadas por heurísticas y ajustes de sus parámetros de exploración.
- Ampliar la evaluación experimental mediante un mayor número de partidas y pruebas contra jugadores humanos o agentes más sofisticados para obtener resultados más robustos.

Referencias

- [1] Russell, S., Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] GeeksforGeeks. (n.d.). *Alpha-Beta Pruning in Adversarial Search Algorithms*. <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>
- [3] Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., Colton, S. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1), 1–43.
- [4] Kocsis, L., Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning*. Proceedings of the 17th European Conference on Machine Learning (ECML 2006).
- [5] Pygame Community. (n.d.). *Pygame documentation*. <https://www.pygame.org/docs/>

...